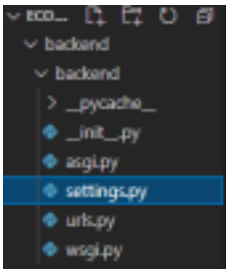
Atelier3 : Django Modèle et interfaçage avec MySql

I. Configuration de base de données MySql

1. Depuis l'interface PhpMyAdmin créer une base de données « dbculinary » 2. Installer mysqlclient, il s'agit d'un connecteur qui vous permet d'utiliser Django avec Mysql.

```
pip install mysqlclient
```

3. Editer le fichier setting.py :



– Commentez la variable DATABASES

– Ajouter le code suivant en dessous de la partie commentée

```
DATABASES = {
 'default': {
 'ENGINE': 'django.db.backends.mysql',
 'NAME': 'dbculinary',
 'USER':'root',
 'PASSWORD':'admin',
 'HOST':'localhost',
 'PORT':'3306',
 }
}
```

4. Exécuter la migration :

```
python manage.py makemigrations
Python manage.py migrate
```

5. Visualiser votre base de données « dbculinary »

II. Création des modèles Django

1. Créer le model Product et Category dans le fichier models.py de l'application « products »

models.py ×

backend > products > models.py > ...

```python
from django.db import models

class Category(models.Model):
    name=models.CharField(max_length=250,unique=True)
    slug=models.SlugField(max_length=50,unique=True)

    def __str__(self):
        return self.name

class Product(models.Model):
    name = models.CharField(max_length=250)
    description = models.TextField()
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    category=models.ForeignKey(Category,on_delete=models.SET_NULL,null=True,related_name='products')

    def __str__(self):
        return self.name
```

2. Appliquer une migration

```
python manage.py makemigrations
Python manage.py migrate
```

3. Créer un super utilisateur : `python manage.py createsuperuser`

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL                                   1: powershell
(myenv) PS C:\Users\user\Documents\Work_2021\cours_2020-2021\sem2_me\django_bensag\doc_formationUdemy\ecommerce\backend> python manage.py createsuperuser
Username (leave blank to use 'user'): hassna
Email address: h.bensag@gmail.com
Password:
Password (again):
Superuser created successfully.
(myenv) PS C:\Users\user\Documents\Work_2021\cours_2020-2021\sem2_me\django_bensag\doc_formationUdemy\ecommerce\backend>
```

4. Lancer le serveur Django. A présent, ouvrir le navigateur et aller à l'URL pour accéder à l'interface Administration de Django :

http://127.0.0.1:8000/admin/

5. Pour rendre les tables (category et product) visible dans l'interface d'administration Django, il faut enregistrer les modèles dans Django Admin, pour cela ouvrez le fichier admin.py de l'application « products »

admin.py

backend > products > admin.py > ...

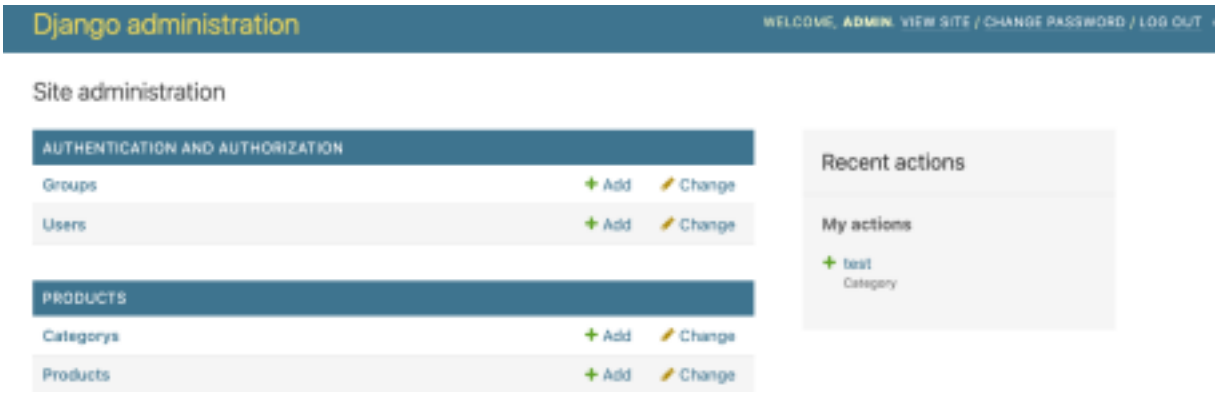
```python
from django.contrib import admin
from .models import Category,Product


@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    prepopulated_fields={'slug':('name',)}


@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display=('name','category','price','stock')
    list_filter=('category',)
    search_fields=('name','description')

```

6. Vérifier l'interface d'administration de Django.



7. Mettre à jour le champ Image du modèle Product :
   - Ajouter un champ image au modèle product

```
models.py ×
backend > products > models.py > ...
1   from django.db import models
2
3   class Category(models.Model):
4       name=models.CharField(max_length=250,unique=True,verbose_name="Nom")
5       slug=models.SlugField(max_length=250,unique=True)
6
7       def __str__(self):
8           return self.name
9
10  class Product(models.Model):
11      name=models.CharField(max_length=250,verbose_name="Nom")
12      description=models.TextField()
13      price=models.DecimalField(max_digits=10,decimal_places=2,verbose_name="Prix")
14      stock=models.PositiveIntegerField(default=0,verbose_name="quantité en stock")
15      category=models.ForeignKey(Category,on_delete=models.SET_NULL,null=True,related_name='products',verbose_name="catégorie")
16      image=models.ImageField(null=True,upload_to='products/')
17
18      def __str__(self):
19          return self.name
```

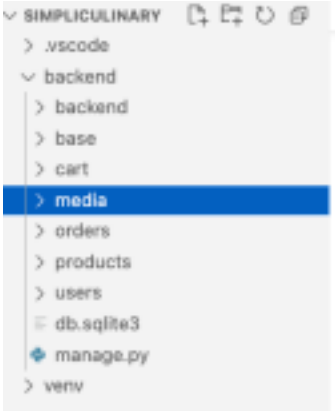
   - Installer **Pillow**, cette bibliothèque est requise pour la gestion des fichiers d'images, par exemple avec les champs ImageField ou FileField

```
pip install pillow
```

   - Faire une migration

III. Configuration des fichiers Media

1. Ajoutez dans la racine de votre projet backend un nouveau dossier appelé « media »

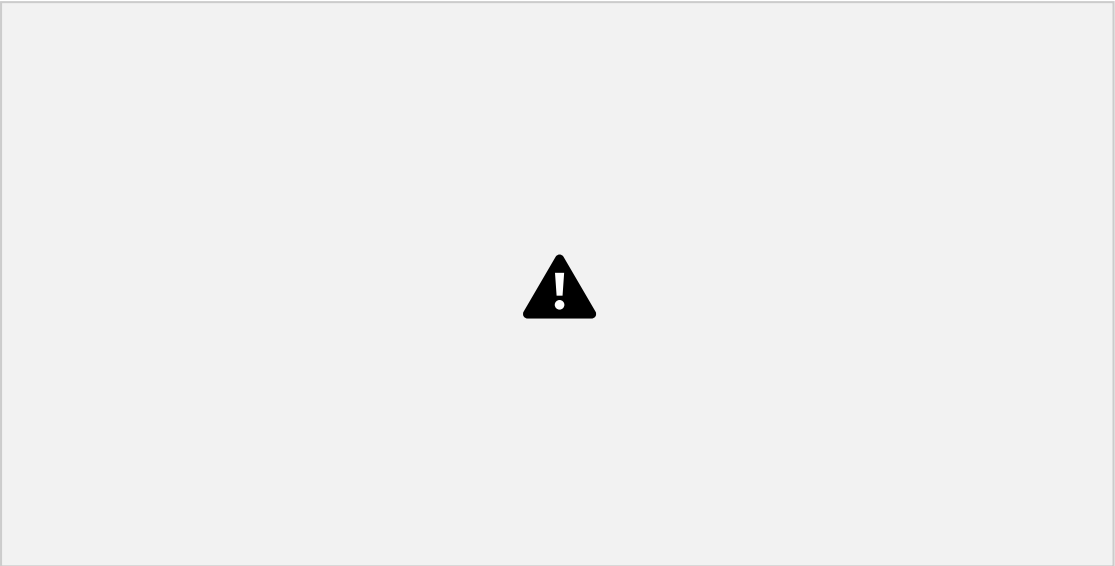


2. Editez le fichier setting.py :

```
settings.py ×
backend > backend > settings.py > ...
118
119   TIME_ZONE = 'UTC'
120
121   USE_I18N = True
122
123   USE_TZ = True
124
125
126   # Static files (CSS, JavaScript, Images)
127   # https://docs.djangoproject.com/en/5.1/howto/static-files/
128
129   STATIC_URL = 'static/'
130
131   MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
132   MEDIA_URL='/media/'
```

<u>Remarque</u> : n'oublier d'importer le module « os », à travers

3. Editez le fichier urls.py du projet backend :



4. Ajoutez un nouveau produit depuis l'interface d'administration et remarquez les changements

IV. Ajouter des jeux de fausses données avec Faker

1. Installer Faker :

```
pip install faker
```

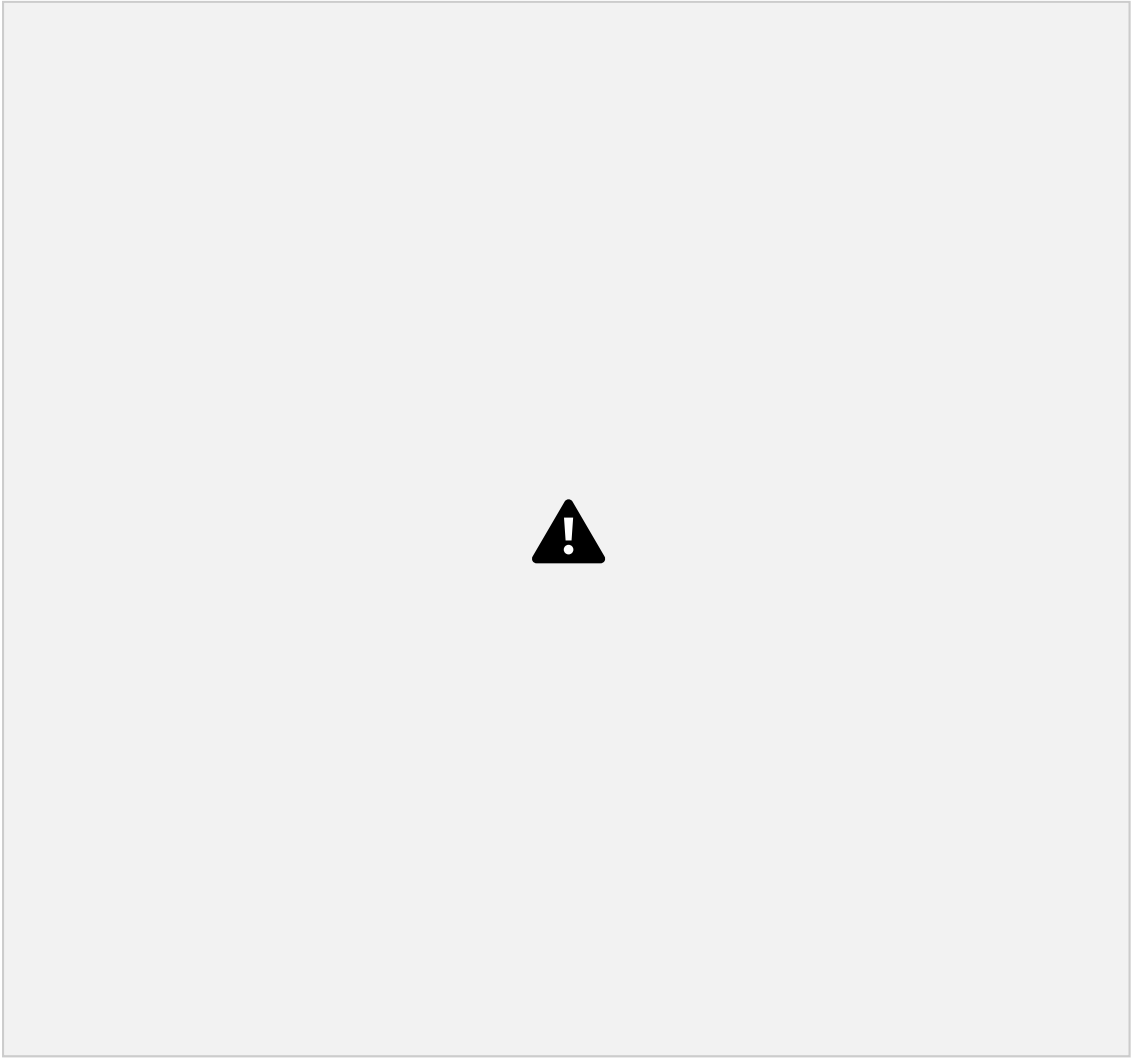
2. Créez un fichier de commande personnalisée :
   - Créez au sein de l'application « products » un répertoire appelé « management » – Puis, créez un dossier « commands »
   - Au sein de ce dernier créer un nouveau fichier appelé « __init__.py »
   - Et un autre fichier appelé « fake_data.py », ensuite tapez le code suivant :

3. Exécuter la commande Django créée

```
python manage.py fake_data
```

Travail à faire :

1. Créer le modèle « CustomUser » dans l'application « users »



2. Enregistrer le modèle auprès de l'interface d'administration

3. Lancez une migration

4. Créez les autres modèles selon le digramme de classe : Cart, CartItem, Order,OrderItem